

COMPUTATIONAL SCIENCES
WINTER TERM 2026
FREIE UNIVERSITÄT BERLIN



Report

**Incompressible Navier Stokes
Project**

Annie, Bianca, Ceren, Gotkug, Jaza, Louis

Gutachter(in):	Dr. Luca Donati
Semester:	Winter term 2026
Verfasser:	Vorname Nachname
Matrikel-Nr.:	1234567
Adresse:	Straße Nr, PLZ Ort
Email:	mail@mail.de
Telefon:	030-123 456 78
Studienfach:	MSc. Computational Sciences

Abgabetermin: 01. Januar 2099

Abstract

TEXT

Contents

List of Figures	iv
List of Tables	v
1 Introduction	2
1.1 Field of relevance	2
1.2 Research goal	2
1.3 Structure of this report	2
2 Methods	3
2.1 First task	3
2.2 Incompressible flow – Chorins projection method	14
2.3 Rayleigh-Bernard convection	17
3 Ergebnisse	20
3.1 Erster Abschnitt	20
3.2 Zweiter Abschnitt	20
3.3 Weitere Abschnitte	20
4 Fazit und Ausblick	22
Bibliography	23

List of Figures

List of Tables

2.1	Discretized Boundary Conditions (BC) Boundary conditions for all of the four boundaries of the twodimensional domain $\Omega = [0, 1]^2$. Each row represents one boundary and shows the continuous and numerical representation of the boundary conditions given in the task. Numerical approximation was performed by using a central finite difference approach.	16
-----	---	----

List of Tables

Clymo (1970)

1 Introduction

Write
Intro-
duction

1.1 Field of relevance

TEXT

1.2 Research goal

TEXT

1.3 Structure of this report

TEXT

2 Methods

2.1 First task

2.1.1 Model

2.1.2 Operator Splitting

Consider a typical PDE of structure

$$\partial_t \mathbf{u} = (A + B + C) \cdot \mathbf{u}$$

Here, $\mathbf{u}$ denotes a vector variable, e.g. the velocity in a fluid at a certain location. A, B, C on the other hand are so called *operators*. Operators are terms in the PDE and can describe certain physical processes. For instance, consider the basic Navier-Stokes equation

$$\partial_t T = \Delta T - \mathbf{u} \nabla T$$

. Here, we could set $A := \Delta, B := -\mathbf{u} \nabla$ and obtain:

$$\partial_t T = (A - B)T$$

A expresses the Laplace operator as a second derivative, B represents advection.

Hence analytical solutions usually don't exist, numerical methods have to be applied to solve such a problem of partial differential equations. As these are often computationally expensive, *operator splitting* provides a procedure to reduce costs for the solution of the PDE by accepting a controlled error. We "split" the problem into subproblems, meaning that we consider each operator individually:

$$\partial_t \mathbf{u} = A\mathbf{u} \tag{2.1}$$

$$\partial_t \mathbf{u} = B\mathbf{u} \tag{2.2}$$

Here, we numerically solve one of the two equations and use the result as the initial

explain
prob-
lem
and
model

conditions for the second to solve it. This is repeated for all timesteps. Of course, this produces an error, as the new initial conditions for the second equation do not completely coincide with the initial conditions we used for the first equation. Thus, solving by integrating the operators subsequently creates an error. Here, different methods of operator splitting exist and different errors are the consequence. In general, the error remains small if the timestep Δt is chosen small in the numerical integration scheme.

First order () operator splitting

include
author

First order operator splitting means to solve both operators subsequently for a given timestep Δt . To further investigate this, consider

$$\partial_t \mathbf{u} = (A + B)\mathbf{u} \quad (2.3)$$

$$\Rightarrow \partial_t \mathbf{u} = A\mathbf{u} \quad (2.4)$$

$$\partial_t \mathbf{u} = B\mathbf{u} \quad (2.5)$$

$$(2.6)$$

with A, B being again some operators. For our problem, these operators will be part of the advection-diffusion equation, as we see later. Note, that we derived two **linear** problems after we applied operator splitting.

That is why we can find simple solutions to the formulated splitted operator problems, namely

$$\mathbf{u}_A(t) = e^{(t-t_0)A}\mathbf{u}_0 \quad (2.7)$$

$$\mathbf{u}_B(t) = e^{(t-t_0)B}\mathbf{u}_0 \quad (2.8)$$

In our case we only want to solve for the next discrete time interval $[t_k, t_{k+1}] = [t_k, t_k + \Delta t]$. We do not solve continuously for the whole time interval. That is why we modify the solutions for the first order operator splitting and rewrite them accordingly to

$$\mathbf{u}_{k+1,A} = e^{(t_{k+1}-t_k)A}\mathbf{u}_{k,A} \quad (2.9)$$

$$\mathbf{u}_{k+1,B} = e^{(t_{k+1}-t_k)B}\mathbf{u}_{k,B} \quad (2.10)$$

With $\Delta t = t_{k+1} - t_k$ we finally get the solution for each operator with respect to the discretized time step Δt (Note that we assume equidistant discretization in time):

2 Methods

$$\mathbf{u}_{k+1,A} = e^{\Delta t A} \mathbf{u}_{k,A} \quad (2.11)$$

$$\mathbf{u}_{k+1,B} = e^{\Delta t B} \mathbf{u}_{k,B} \quad (2.12)$$

For first-order operator splitting the following algorithm is applied to the abstract problem $\partial_t \mathbf{u} = (A + B)\mathbf{u}$. The equation is split as shown into the two separate operator problems:

1. Set $\mathbf{u}_{k,A} := \mathbf{u}_k$
2. Solve $\mathbf{u}_{k+1,A} = e^{\Delta t A} \mathbf{u}_{k,A}$ on the next interval $[t_k, t_k + \Delta t]$.
3. We obtain $\mathbf{u}_{k+1,A}$.
4. Consider $\mathbf{u}_{k+1,A}$ as the new initial condition in the second operator equation: $\mathbf{u}_{k,B} := \mathbf{u}_{k+1,A}$
5. Solve $\mathbf{u}_{k+1,B} = e^{\Delta t B} \mathbf{u}_{k,B} = e^{\Delta t B} \mathbf{u}_{k+1,A}$.
6. We obtain $\mathbf{u}_{k+1,B}$
7. We consider $\mathbf{u}_{k+1,B}$ as our solution for $\mathbf{u}_{k+1}$

Written in a compact manner, we obtain

$$u_{k+1} = e^{\Delta t B} e^{\Delta t A} u_k$$

as our first-order operator splitting procedure. Like already mentioned, this algorithm relies on the assumption, that for small timesteps Δt only small difference remains between $u_{k+1,A}$ and $u_{k+1,B}$ when evaluated subsequently. This algorithm is simple and easy to implement. Nevertheless, the GTE error is of order $\mathcal{O}(\Delta t)$ () and can be further reduced. This leads to the second-order operator methods.

Second order operator splitting (Strang-splitting)

To reduce the GTE of the operator splitting scheme to an error of order $\mathcal{O}(\Delta t^2)$ the following can be applied to approximate the solution $\mathbf{u}_{k+1}$ at the next timestep $t_{k+1} = t_k + \Delta t$:

$$\mathbf{u}_{k+1} = e^{\frac{\Delta t}{2} A} \cdot e^{\Delta t B} \cdot e^{\frac{\Delta t}{2} A} \mathbf{u}_k \quad (2.13)$$

include
some
computational
efficiency
stuff,
maybe
even
related
to our
testing?

include
source

2.1.3 The velocity equation

We have seen operator splitting for a rather general problem. We now apply operator splitting in the context of the velocity equation from the Navier-Stokes equation

$$\partial_t \mathbf{u} + \mathbf{u} \cdot \nabla \mathbf{u} + \frac{1}{\rho} \nabla p = \nu \Delta \mathbf{u} + \beta T \mathbf{e}_z$$

and obtain three different operators:

1. Advection: $\partial_z \mathbf{u} = \mathbf{u} \cdot \nabla \mathbf{u}$
2. Diffusion + forcing: $\partial_t \mathbf{u} = \nu \Delta \mathbf{u} + \beta T \mathbf{e}_z$
3. Projection step (Incompressibility): $\mathbf{u}^{k+1} = \mathbf{u}^* - \Delta \mathbf{p}$, with $\nabla \cdot \mathbf{u}^{k+1} = 0$
The pressure acts like a Lagrangian multiplier and enforces $\nabla \cdot \mathbf{u} = 0$

To approach this problem of increased complexity we divide the steps to compute a numerical solution for the velocity. Each step assumes different simplifications and approximations to ensure all the demanded properties for a solution of the vector field of velocity.

1. Step 1:

Ignore Incompressibility: $\partial_t \mathbf{u} + \mathbf{u} \cdot \nabla \mathbf{u} = \nu \Delta \mathbf{u} + f$. Here, we obtain the intermediate velocity $\mathbf{u}^* \Rightarrow \nabla \cdot \mathbf{u} \neq 0$

2. Step 2:

We now use this intermediate velocity to respect incompressibility and include pressure:

$$\mathbf{u}^{k+1} = \mathbf{u}^* - \nabla p$$

We use the *Helmholtz decomposition* to do this:

$$\mathbf{u}^* = \mathbf{u}^\perp + \nabla \phi$$

It follows immediately, that $\mathbf{u}^{k+1} = \mathbf{u}^\perp$. We want

$$\nabla \cdot \mathbf{u}^{k+1} = 0 \Rightarrow \nabla \cdot (\mathbf{u}^* - \nabla p) = 0 \Rightarrow \nabla \cdot (\mathbf{u}^* - \nabla p) = 0 \Rightarrow \nabla \cdot \mathbf{u}^* = \Delta p$$

as this gives us the Poisson equation for pressure.

2 Methods

Here, we want to quickly summarize what has happened so far. If the reader does not feel in need of an overview, the following paragraphs can easily be left out. Nevertheless, we want to give a quick impression of the actual problem at this point to make it easier to follow later on. We begin by giving context to the physical problem addressed by Navier-Stokes.

The Navier-Stokes equations are partial differential equations that are hard to solve analytically. Thus, numerical methods are necessary to solve for the velocity on our domain. The general problem described by the Navier-Stokes can be imagined in the following way: We ensure some dynamical characteristics that our fluid should have (e.g. how the volume changes under pressure, how the flow at one location is influenced by other locations, etc.). These properties are ensured by the different operators that are all related to the flow behaviour in time. So basically, we are interested in the velocity of the fluid at each time step at every location on our domain. And the temporal development of such velocity at a certain location is determined by spatial characteristics at this location (e.g. the first and second derivative in space compared to the surrounding fluid). Basically, the *temporal* change is driven by *spatial* derivatives of the velocity itself. Advection, for instance, is ensuring the transport of the fluid's mass along its own direction of flow. Diffusion on the other hand is modelling the smoothing of differences in velocity at different locations with time. Thus, if no additional forcing or energy input would cause unevenly distributed velocity behaviour, we should see stillness in the motion of the fluid. These processes only depend on characteristics of the vector field of the velocity $\mathbf{u}$. But other terms, like the forcing βT_z and $\frac{1}{\rho}\nabla p$ are independent of the velocity $\mathbf{u}$ and create additional complexity. Although we will ignore any additional forcing (you could imagine heating at a certain location that e.g. creates convection) later on, we still are left to deal with the pressure properties.

In our case, we consider an incompressible fluid. Formally this is ensured by $\nabla \mathbf{u} = 0$. Intuitively, that means that although we apply force to the fluid it will **not** change the volume it occupies in space. Vice versa, for decreased pressure there will not be any spatial expansion of the fluid. So at no point in time and space, volume should be 'lost' or 'gained'. As the pressure in a flowing fluid is a dynamical variable, we have to solve for it, too. Not only is it dynamical, it is also coupled to the velocity $\mathbf{u}$.

Why is this a problem? If we were to solve a PDE that only depends on $\mathbf{u}$ we could rather easily discretize and solve numerically for $\mathbf{u}$. But as we have another dynamical variable p this gets more complicated. This is why we need two steps: First, we assume that there is no coupling between velocity $\mathbf{u}$ and pressure p . We also neglect $\nabla \mathbf{u} = 0$ in this step. The first step therefore yields a first interim result $\mathbf{u}^*$ for the velocity for the next timestep (at each

location). In the second step we then include pressure and ensure the incompressibility. It turns out that we can efficiently combine this property with our pressure operator if a *Helmholtz* decomposition is applied to the vector field. Finally, this leads to a poisson equation to ensure the incompressibility and obtain a solution for the pressure.

This way, we successfully obtained numerical solutions for an intermediate velocity and pressure that respects the incompressibility. The last challenge remaining is to obtain a discrete next step for a velocity that also respects the incompressibility. A description of such a procedure will be done in chapter 2.2.

Operator splitting for the velocity equation

Coming back to our operators of the velocity equation, we want to apply operator splitting for all the different terms. We begin with the advection operator term.

$$\partial_t \mathbf{u} + a_y \partial_y \mathbf{u} + a_z \partial_z \mathbf{u} = 0$$

Here, we defined $\mathbf{a} = \begin{pmatrix} a_y(y, z) \\ a_z(y, z) \end{pmatrix}$ and $\mathbf{u} = \begin{pmatrix} v \\ w \end{pmatrix}$

The actual splitting is now executed in space first. The two spatial operators for each spatial dimension lead to the following approach:

1. Solve

$$\partial_t \mathbf{u} + a_y \partial_y \mathbf{u} = 0$$

Or, more explicitly written:

$$\begin{cases} \partial_t v + a_y \partial_y v &= 0 \\ \partial_t w + a_y \partial_y w &= 0 \end{cases}$$

We do the same for the second spatial dimension:

2. Solve

$$\partial_t \mathbf{u} + a_z \partial_z \mathbf{u} = 0$$

Again, more explicitly written:

$$\begin{cases} \partial_t v + a_z \partial_z v &= 0 \\ \partial_t w + a_z \partial_z w &= 0 \end{cases}$$

3. Apply Lie Splitting in time: $\mathbf{u}^{k+1} = A_z(\Delta t)A_y(\Delta t)\mathbf{u}^k$

We now obtained a operator splitting for the (linear) advection equation. In the next part we will examine the upward finite difference method to solve the obtained equations numerically.

2.1.4 Upward Finite Difference Method

To introduce the Upward Finite Difference Method we first want to look again at the one-dimensional linear advection equation

$$\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$$

Intuitively, this equation describes a wave that propagates along the x -axis with velocity a . Now imagine the x -axis laid out horizontally in front of you. If $a > 0$, the wave propagates to the right. Thus, an upwind direction would correspond to the direction in the left – towards smaller values. For $a < 0$ this changes accordingly: The upwind direction corresponds to the direction of higher x -values, so to the right.

That is why we will use backward finite difference methods for $a > 0$ and forward finite difference methods for $a < 0$. We will use the upwind discretization in a onedimensional case and therefore discretize the grid in the following manner:

$$\begin{aligned} x_i &:= i \Delta x, \quad i = 0, \dots, N \\ t^n &:= n \Delta t \end{aligned}$$

As already described, we will use different methods depending on the exact value of a to ensure the upwind finite difference scheme.

1. $a > 0$ (backward):

$$\partial_x \mathbf{u} \approx \frac{\mathbf{u}_i - \mathbf{u}_{i-1}}{\Delta x}$$

2. $a < 0$ (forward):

$$\partial_x \mathbf{u} \approx \frac{\mathbf{u}_{i+1} - \mathbf{u}_i}{\Delta x}$$

After executing the step for the spatial dimensions, we proceed with the discrete step in time:

$$\frac{\mathbf{u}_{i+1}^n - \mathbf{u}_i^{n+1}}{\Delta t} = -a(\partial_x \mathbf{u})_i^n$$

also
define
discrete
no of
steps
for
Time?
So
maybe
 N_x and
 N_t ?

2 Methods

1. $a > 0$ (backward):

$$\frac{\mathbf{u}_{i+1}^n - \mathbf{u}_i^{n+1}}{\Delta t} = -a(\partial_x \mathbf{u})_i^n \quad (2.14)$$

$$\Rightarrow \mathbf{u}_i^{n+1} - \mathbf{u}_i^n = -a \frac{\Delta t}{\Delta x} (\mathbf{u}_i^n - \mathbf{u}_{i-1}^n) \quad (2.15)$$

$$\Rightarrow \mathbf{u}_i^{n+1} = \left(1 - a \frac{\Delta t}{\Delta x}\right) \mathbf{u}_i^n + a \frac{\Delta t}{\Delta x} \mathbf{u}_{i-1}^n \quad (2.16)$$

$$\Rightarrow \mathbf{u}_i^{n+1} = A_1 \mathbf{u}_i^n \quad (2.17)$$

2. $a < 0$ (forward):

$$\frac{\mathbf{u}_{i+1}^n - \mathbf{u}_i^{n+1}}{\Delta t} = -a(\partial_x \mathbf{u})_i^n \quad (2.18)$$

$$\Rightarrow \mathbf{u}_i^{n+1} - \mathbf{u}_i^n = -a \frac{\Delta t}{\Delta x} (\mathbf{u}_{i+1}^n - \mathbf{u}_i^n) \quad (2.19)$$

$$\Rightarrow \mathbf{u}_i^{n+1} = \left(1 - a \frac{\Delta t}{\Delta x}\right) \mathbf{u}_i^n - a \frac{\Delta t}{\Delta x} \mathbf{u}_{i+1}^n \quad (2.20)$$

$$\Rightarrow \mathbf{u}_i^{n+1} = A_2 \mathbf{u}_i^n \quad (2.21)$$

Thus, we obtained our operator for each upwind case which depends only on the sign of a . To shorten the emerging equations for all of our discretized locations we will write the operator in a compact matrix notation.

$$A_1 = \begin{pmatrix} \left(1 - a \frac{\Delta t}{\Delta x}\right) & 0 & \dots & 0 \\ a \frac{\Delta t}{\Delta x} & \ddots & & \vdots \\ 0 & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & \dots & 0 & a \frac{\Delta t}{\Delta x} & \left(1 - a \frac{\Delta t}{\Delta x}\right) \end{pmatrix}$$

$$A_2 = \begin{pmatrix} \left(1 - a \frac{\Delta t}{\Delta x}\right) & -a \frac{\Delta t}{\Delta x} & 0 & \dots & 0 \\ 0 & \ddots & \ddots & & \vdots \\ & & \ddots & \ddots & 0 \\ \vdots & & & \ddots & -a \frac{\Delta t}{\Delta x} \\ 0 & \dots & 0 & \left(1 - a \frac{\Delta t}{\Delta x}\right) \end{pmatrix}$$

Stability of the Operators

In general, the stability condition for a matrix operator A is given by $\|A\|_\infty \leq 1$. We will investigate our both operators A_1 (corresponds to $a > 0$ and therefore backward finite difference method) and A_2 (corresponds to $a < 0$ and therefore forward finite difference method) with respect to this stability condition.

$$\|A_1\|_\infty = \left| a \frac{\Delta t}{\Delta x} \right| + \left| 1 - a \frac{\Delta t}{\Delta x} \right| \leq 1 \quad (2.22)$$

$$\Rightarrow^{a>0} \left| 1 - a \frac{\Delta t}{\Delta x} \right| \leq 1 - a \frac{\Delta t}{\Delta x} \quad (2.23)$$

$$\Rightarrow 1 - a \frac{\Delta t}{\Delta x} \geq 0 \quad (2.24)$$

$$\Rightarrow a \frac{\Delta t}{\Delta x} \leq 1 \quad (2.25)$$

Repeating this for our second operator A_2 we get

$$\|A_2\|_\infty = \left| a \frac{\Delta t}{\Delta x} \right| + \left| 1 - a \frac{\Delta t}{\Delta x} \right| \leq 1 \quad (2.26)$$

$$\Rightarrow^{a>0} \left| 1 - a \frac{\Delta t}{\Delta x} \right| \leq 1 + a \frac{\Delta t}{\Delta x} \quad (2.27)$$

$$\Rightarrow 1 + a \frac{\Delta t}{\Delta x} \geq 0 \quad (2.28)$$

$$\Rightarrow -a \frac{\Delta t}{\Delta x} \leq 1 \quad (2.29)$$

and, thus, obtain

$$\left| a \frac{\Delta t}{\Delta x} \right| \leq 1$$

as our general stability condition for our operator.

2.1.5 Twodimensional upwind scheme with operator splitting

We now want to jointly apply operator splitting and the upwind finite difference method on our twodimensional advection equation. Therefore, we define $u_{ij}^n := u(t^n, y_i, z_j)$ and assume an equidistant grid as the discretized version of our problem. Therefore we declare Δy and Δz as the distances between discrete grid points in space.

We first want to solve the advection term in the y direction. We fix a value z_j and solve

2 Methods

the resulting equation

$$\partial_t \mathbf{u} + a_y \partial_y \mathbf{u} = 0$$

Whether forward or backward finite difference method is applied depends on a again:

$$(\partial_y \mathbf{u})_{ij} = \begin{cases} \frac{u_{ij} - u_{i-1,j}}{\Delta y} & a_y > 0 \\ \frac{u_{i+1,j} - u_{ij}}{\Delta y} & a_y < 0 \end{cases}$$

As a result of our first operator we obtain

$$u_{ij}^* = u_{ij}^n - \Delta t a_{y,ij} (\partial_y \mathbf{u})_{ij}$$

where we ensure

$$a_y \frac{\Delta t}{\Delta y} \leq 1$$

. We use this result to proceed with the solution of the z -coordinate corresponding to the second operator:

$$\partial_t \mathbf{u} + a_z \partial_z \mathbf{u} = 0$$

We define analogously

$$(\partial_z \mathbf{u}^*)_{ij} = \begin{cases} \frac{u_{ij}^* - u_{i-1,j}^*}{\Delta y} & a_y > 0 \\ \frac{u_{i+1,j}^* - u_{ij}^*}{\Delta y} & a_y < 0 \end{cases}$$

Using the result from the first splitted operator we are now able to compute

$$u_{ij}^{n+1} = u_{ij}^* - \Delta t a_{z,ij} (\partial_z \mathbf{u}^*)_{ij}$$

We ensure stability in this case as well by respecting $\frac{|a_z| \Delta t}{\Delta z} \leq 1$.

We can now derive a stability condition for all of our discretized points in space. We first obtain

$$\max |a_y| = \max_{i,j} |a_y(y_i, z_j)|$$

and

$$\max |a_z| = \max_{i,j} |a_z(y_i, z_j)|$$

. By the help of these variables we can now deduce the overall constraint for Δt , that is necessary to ensure stability:

$$\Delta t \leq \min \left(\frac{\Delta y}{\max |a_y|}, \frac{\Delta z}{\max |a_z|} \right)$$

2 Methods

Summarized, we successfully implemented operator splitting and upwind finite difference method for the linear advection term and ensured stability all over our domain.

boundary
condi-
tions
imple-
menta-
tion

2.2 Incompressible flow – Chorins projection method

We consider the Navier-Stokes equations for velocity

$$\partial_t \mathbf{u} + \mathbf{u} \cdot \nabla \mathbf{u} + \frac{1}{\rho} \nabla p = \nu \Delta \mathbf{u} \quad \text{with } \beta = 0 \quad (2.30)$$

$$\nabla \cdot \mathbf{u} = 0 \quad (2.31)$$

Here, we implement the boundary condition

$$\mathbf{u}(t, 0, z) = \mathbf{u}(t, 1, z) \quad (2.32)$$

$$\mathbf{w}(t, y, 0) = \mathbf{w}(t, y, 1) = 0 \quad \forall t \in [0, t_{end}], (y, z) \in \Omega \quad (2.33)$$

In order to solve this differential equation we use a projection method. . As demanded by the project task we implement Chorins projection method. Conceptually, this approach can be split into two steps:

explain
why
we use
a pro-
jection
method

1. We calculate an intermediate velocity that does not satisfy the incompressibility constraint ($\nabla \cdot \mathbf{u} = 0$). This intermediate velocity is computed at each time step (ignoring the pressure term)
2. Subsequently, the pressure p is used to *project* the intermediate velocity onto a space of divergent free velocity field.

Formally, we perform

1.

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = -(\mathbf{u}^n \cdot \nabla) \mathbf{u}^n + \nu \nabla^2 \mathbf{u}^n$$

2.

$$\frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} = -\frac{1}{\rho} \nabla p^{n+1}$$

Two things are especially noteworthy about this procedure. Firstly, the pressure p is time-invariant. That means, that the pressure p is unaffected by our timestep Δt . Secondly, we can summarize that this procedure is a kind of operator splitting.

But we are not finished here. In our first step of the described scheme we obtain the intermediate velocity $\mathbf{u}^*$. However, this velocity vector is not complying with our constraint of zero divergence. Additionally, the solution of the second equation of this scheme requires

2 Methods

knowledge about p . That is why we rewrite the second equation:

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla p^{n+1}$$

Ensuring the absence of divergence ($\nabla \mathbf{u}^{n+1} = 0$) allows us to further extend this equation:

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla p^{n+1} \quad (2.34)$$

$$\text{respecting divergence} \quad 0 = \nabla \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla^2 p^{n+1} \quad (2.35)$$

$$\Rightarrow \Delta p^{n+1} = \frac{\rho}{\Delta t} (\nabla \mathbf{u}^*) \quad (2.36)$$

The last result reflects the Poisson equation for pressure.

We can see that the method is called a 'projection' because we decompose $\mathbf{u}^*$ into a divergence-free (solenoidal) part $\mathbf{u}_{\text{sol}}$ and irrotational part $\mathbf{u}_{\text{irrot}}$. Then with the projection step $\mathbf{u}_{\text{irrot}} = \nabla \phi$ is removed so that the remaining part is negative divergence-free.

$$\mathbf{u} = \mathbf{u}_{\text{sol}} + \mathbf{u}_{\text{irrot}} = \mathbf{u}_{\text{sol}} + \nabla \phi$$

Since $\nabla \mathbf{u}_{\text{sol}} = 0$ the whole expression simplifies when we calculate the divergence of the total expression:

$$\Delta \mathbf{u} = \Delta \phi$$

Here, $\Delta = \nabla^2$ represents the Laplacian operator. Consequently, the solenoidal part can be calculated as $\mathbf{u}_{\text{sol}} = \mathbf{u} - \nabla \phi$.

This kind of projection method is called Helmholtz-Hodge decomposition.

We once again consider the Poisson equation we want to solve:

$$\Delta p = \frac{\rho}{\Delta t} (\nabla \mathbf{u}^*) \quad \text{in the domain } \Omega = [0, 1]^2$$

If we assume Neumann boundary conditions it leads to

$$\frac{\partial p}{\partial \mathbf{n}} = 0 \Leftrightarrow \nabla p \cdot \mathbf{n} = 0 \quad \text{on } \partial \Omega$$

With this boundary condition we get a solution that is not unique as p is defined up to an additive constant. To solve the Poisson equation we equidistantly discretize the spatial domain.

$$y_i = i \Delta y \quad i = 0, \dots, N_y - 1$$

Boundary	BC	numerical BC	Resulting condition
Bottom($z = 0, j = 0$)	$\frac{\partial p}{\partial z} = 0$	$\frac{p_{i,1} - p_{i,-1}}{2\Delta z} = 0$	$p_{i,1} = p_{i,-1}$
Top($z = 1, j = N_z - 1$)	$\frac{\partial p}{\partial z} = 0$	$\frac{p_{i,N_z} - p_{i,-2}}{2\Delta z} = 0$	$p_{i,N_z} = p_{i,N_z-2}$
Left($y = 0, i = 0$)	$\frac{\partial p}{\partial y} = 0$	$\frac{p_{1,j} - p_{-1,j}}{2\Delta z} = 0$	$p_{1,j} = p_{-1,j}$
Right($y = 1, i = N_y - 1$)	$\frac{\partial p}{\partial y} = 0$	$\frac{p_{N_y,j} - p_{N_y-2,j}}{2\Delta z} = 0$	$p_{N_y,j} = p_{N_y-2,j}$

Table 2.1: **Discretized Boundary Conditions (BC)** Boundary conditions for all of the four boundaries of the twodimensional domain $\Omega = [0, 1]^2$. Each row represents one boundary and shows the continuous and numerical representation of the boundary conditions given in the task. Numerical approximation was performed by using a central finite difference approach.

$$z_j = j\Delta z \quad j = 0, \dots, N_z - 1$$

To approximate the second derivatives (Laplace operator) we implement the second finite difference in two dimensions:

$$\Delta p = \nabla^2 p = \frac{\partial^2 p}{\partial y^2} + \frac{\partial^2 p}{\partial z^2} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta y^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta z^2}$$

2.2.1 Discretized Poisson equation

We now plug the discretized Laplace operator into our Poisson equation. Remember that the Poisson we want to solve was

$$\Delta p^{n+1} = \frac{\rho}{\Delta t} (\nabla \mathbf{u}^*)$$

Thus, plugging in the numerical approximations and rearranging yields:

$$-2\left(\frac{1}{\Delta y^2} + \frac{1}{\Delta z^2}\right)p_{i,j} + \frac{p_{i+1,j} + p_{i-1,j}}{\Delta y^2} + \frac{p_{i,j+1} + p_{i,j-1}}{\Delta z^2} = \frac{\rho}{\Delta t} (\nabla \mathbf{u}^*)_{i,j}$$

We also have to ensure, that our numerical solution respects the boundary conditions. We will use a central finite difference approach for this. The central finite difference can generally be written as

$$f'_i \approx \frac{f_{i+1} - f_{i-1}}{2\Delta x}$$

Each boundary of our domain then corresponds to another numerical condition to be met which can be seen in table 2.2.1.

2.2.2 Numerical procedure

Summarising, we obtain the following numerical procedure:

1. Solve $\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = -(\mathbf{u}^n \cdot \nabla) \cdot \mathbf{u}^n + \nu \Delta \mathbf{u}^n$ with a suitable integrator scheme
2. Obtain p by solving the discretized Poisson equation
3. Solve $\frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} = -\frac{1}{\rho} \nabla p$ and obtain $\mathbf{u}^{n+1}$

2.3 Rayleigh-Bernard convection

So far we have ignored any forcing in the velocity equation of the Navier-Stokes equation. In the following section we want to introduce thermal forcing to reproduce the Rayleigh-Bernard convection cells. Just to name one example, they occur in climatology on different scales and convective phenomena are important in various processes all over the global atmospheric system. In this section we want to find adequate initial conditions and parameters to reproduce such a convective behaviour using our solutions.

Physical parameters

To start, we once again want to consider the momentum equation. For our problem, we will use the Boussinesq-approximation . This leads to

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} + \frac{1}{\rho} \nabla p = \nu \nabla^2 \mathbf{u} + T \beta e_z$$

explain
Boussinesq
approximation

As we are interested in a certain set of parameters we briefly want to give some intuitive idea of what the parameters are describing. Consider ν for example: If ν is too high (modelling too much friction), the fluid will remain in a purely conductive state, implying the absence of convective cells. The buoyancy parameter β is of importance, too. If β is too low, then heat will simply diffuse across the fluid instead of creating motion due to localized heat forcing. We see, that these parameters are essential for a successful recreation of the convective cells. While viscosity ν determines, in how far the fluid is 'allowed' to react to the thermal forcing β represents the strength of the thermal coupling.

In a dimensionless, more general context, the onset of convection is determined by the Rayleigh number. In our case, the Rayleigh number is directly proportional to $\frac{\beta}{\nu}$. We have to choose β large enough (or ν small) to overcome the internal friction and thermal diffusion of the fluid.

2.3.1 Initial conditions

To trigger the desired convection cells we not only are in need of an accurate parameterization. We also rely on appropriate initial conditions. As symmetry in the initial conditions could hinder the formation of convective cells, we can not start with a perfectly uniform or stratified fluid.

Instead, we are suggesting the following characteristics for the initial conditions in our context. Firstly, we want to implement a conductive profile. That means, that we start with a temperature field $T_0(y, z)$ that roughly follows the linear profile $1 - z$ (hot at the bottom, cold at the top).

Secondly, we want to break the symmetry in the velocity field by the introduction of small perturbations. We chose random noise that is used to initialize the velocity field $\mathbf{u}_0(y, z)$. These perturbations will be amplified by the buoyancy term $T\beta e_z$ and, if the Rayleigh number is high enough, eventually lead to the emergence of Rayleigh-Bernard convection.

Formally, the described set of initial conditions can be described in the following way. On our domain $\Omega = [0, 1]^2$ we introduce time-invariant temperatures (constant heating across time): In a perfectly still fluid ($\mathbf{u} = 0$) the temperature settles into a linear conductivity profile.

$$T(x, y) = 1 - z$$

This set up implies $T = 1$ at $z = 0$ (bottom) and $T = 0$ at $z = 1$ (top). Note, that this state satisfies the equilibrium condition meaning

$$\partial_t T + \mathbf{u} \Delta T = \Delta T$$

where $\Delta T = 0$ and $\mathbf{u} \cdot \Delta t = 0$.

To introduce the mentioned perturbations in the temperature field, we will add noise to these initial conditions.

$$T = (1 - z) + \epsilon$$

To understand, if such perturbations will grow or vanish, we substitute the temperature field into the momentum equations and linearize them. This is performed by dropping nonlinear terms like $\nu \cdot \nabla \epsilon$ or $(\mathbf{u} \cdot \nabla) \mathbf{u}$

2 Methods

That is how we obtain the temperature equations

$$\partial_t + \mathbf{u} \cdot \nabla (1 - z) = \Delta \epsilon \quad (2.37)$$

$$\nabla (1 - z) = -e_z \quad (2.38)$$

$$(2.39)$$

This leads to $\mathbf{u} \cdot (1 - z) = -w$ So in total we obtain the linearized heat equation

$$\partial_t \epsilon = w + \Delta \epsilon$$

We can see, that a vertical upward motion $w > 0$ acts as the source term that increases the local temperature perturbation.

If we ignore non-linear terms in the momentum equation and split decompose pressure into a static part and perturbation p' , the vertical component of the momentum becomes:

$$\partial_t w + \frac{1}{\rho} \partial_z p' = \nu \Delta w + \epsilon \beta$$

Here, $\epsilon \beta$ is the buoyancy force. It drives the vertical velocity w , which in turn increases the perturbation ϵ . Thus, stability is depending on buoyancy and damping.

The perturbations depend on two major forces:

1. **Driving forces:** The buoyancy term ($T\beta e_z$) which amplifies the disturbances as hot fluid is rising.
2. **Damping forces:** The viscosity $\nu \Delta \mathbf{u}$ and thermal diffusion ΔT , which act to 'smooth' velocity and temperature gradients.

In conclusion, we see that the growth rate of the perturbations ϵ become positive only when the buoyancy term overcomes these damping effects. If β is too small, diffusion is more important, and the perturbation decays back to the conductive state.

3 Ergebnisse

Results,
Plots

3.1 Erster Abschnitt

3.2 Zweiter Abschnitt

3.3 Weitere Abschnitte ...

4 Fazit und Ausblick

(...)

Bibliography

Clymo, R. S. (1970). The Growth of Sphagnum: Methods of Measurement. *Journal of Ecology*, 58(1):13–49. Publisher: [Wiley, British Ecological Society].